Practical 07 Logical Reasoning and Inference

SE3062 Intelligent Systems
Part 5 Theory to Implementation Mapping

Student name: I.D.P Ashen

Student ID: IT24610781

1. Reachability vs Feasibility

In Step 3.2, you modified the A algorithm's neighbor-generation loop. How does your new code differentiate between checking if a node is "Reachable" (like a wall collision) versus checking if it is "Feasible"? Explain how the Knowledge Base enforces feasibility.*

Reachability concerns physical access: the agent must stay inside the grid and cannot move through walls. In my implementation, successors() checks these conditions before yielding a neighbouring tile. Feasibility adds a logical safety check. A tile may be physically accessible but still be rejected because its facts imply that the agent should retreat.

For each candidate neighbour, A* calls clear_facts(), adds that tile's facts, and runs forward_chain(). If Retreat is inferred, the continue statement skips the neighbour before it enters the frontier. For tile (1, 0), TargetVisible and HasDust imply SafeToEngage; together with BloodseekerMissing, this implies Retreat. The integration test therefore finds the alternative path Up, Right, Right, Down. Clearing facts between tiles prevents conclusions from one tile affecting another. Under the worksheet's rule, a tile is allowed when Retreat is not inferred; this is not a proof that an unknown tile is safe.

2. Declarative vs Procedural Paradigms

Why did we build a KnowledgeBase class with a forward_chain() loop instead of simply writing if 'TargetVisible' in percepts and 'HasDust' in percepts: directly inside the agent's movement code? What happens to the agent's code complexity if we add 50 new game rules?

The knowledge base separates what the agent knows from how it moves. Facts and rules are stored as data, while one general inference loop applies them. This also supports chains of deductions: SafeToEngage can be derived first and then used to infer Retreat. A direct if statement can handle one condition, but many interacting conditions would make the movement code harder to read, update and test.

If 50 new rules are added, they can be registered through tell_rule() without adding 50 separate conditions to A*. The movement code can keep the same check for Retreat, provided the new rules use that decision convention. This reduces maintenance complexity, but it does not make inference free: the engine must check more rules and may need additional passes. New rules still need testing for their intended interactions.

3. Modus Ponens and Horn Clauses

The rule ['TargetVisible', 'HasDust'] implies 'SafeToEngage' is formally known as a Horn Clause. Briefly explain how your Python implementation of the forward_chain() while loop acts as a mathematical execution of the Modus Ponens inference rule.

Modus Ponens states that if a premise P is true and P implies Q, then Q can be concluded. Here, P is the conjunction TargetVisible AND HasDust, and Q is SafeToEngage. The expression all(premise in self.facts for premise in premises) checks that every required premise is known. When the check succeeds, the engine adds the rule's conclusion to the facts set.

This rule is a definite Horn clause: it has a conjunction of positive premises and one positive conclusion. Newly inferred facts can trigger other rules, so the while loop repeats until a complete pass adds nothing

new. Checking that the conclusion is not already present prevents repeated additions. With the finite facts and rules used here, this reaches a fixed point and terminates. The reversed-rule test confirms that Retreat is still inferred even when its rule is checked before the rule that produces SafeToEngage.